Supermarket route optimizer

A multi-criteria optimization project

Supervised by: Dr. Ir. Anh Vu Doan
Chair/Department: Chair of Integrated Systems (Prof. Herkersdorf)
Submitted by: Timo Matuszewski
GitHub-Project: https://github.com/mangam-02/supermarket-route-optimizer
Submission date: 22.03.2026

Appendix I

I hereby declare that I have written the submitted thesis independently. While I have used AI tools to assist with formulation and phrasing, all ideas, analyses, and conclusions presented in this thesis are entirely my own. Furthermore, the methods applied in this work are based on the course "Multi-Criteria Optimization and Decision Analysis for Embedded Systems Design" at the Technical University of Munich, as well as additional resources researched from publicly available sources on the internet.

Munich, 22.03.2026, Signature

Contents

1 Introduction **5**

2 Supermarket representation and Setup **6**

2.1 Supermarket Modeling . . . 6

2.2 Shopping List Structure . . . 7

3 Walking Distance Optimization **8**

3.1 Walking Cost Matrix for Product Categories . . . 8

3.2 Product-Specific Path Matrices . . . 8

3.3 Metaheuristic Approaches for Shopping Route . . . 9

3.3.1 Brute-Force Heuristic . . . 9

3.3.2 Local Search Heuristic . . . 11

3.3.3 Extended Neighborhood: Pairwise Swaps . . . 11

3.3.4 GRASP (Greedy Randomized Adaptive Search Procedure) . . . 13

4 Product Weight Optimization **15**

5 Product Hardness Optimization **17**

6 Multi-Criteria Optimization **18**

6.1 NSGA-II for Bi-Objective Optimization . . . 18

6.1.1 Problem Formulation . . . 18

6.1.2 Pareto Dominance . . . 18

6.1.3 Crowding Distance . . . 19

6.1.4 Initializing Population . . . 19

6.1.5 Selection, Crossover and Mutation . . . 20

6.1.6 Replacement Strategy . . . 20

6.1.7 Duplicate Removal . . . 20

6.1.8 Hypervolume Indicator . . . 21

6.1.9 Results . . . 22

6.1.10 Decision Making . . . 22

6.1.11 Learning . . . 23

6.2 NSGA-II for Three-Objective Optimization . . . 23

6.2.1 Problem Formulation . . . 23

6.2.2 Pareto Dominance . . . 23

6.2.3 Crowding Distance . . . 24

6.2.4 Initialization, Genetic Operators and Replacement . . . 24

6.2.5 Hypervolume Indicator . . . 24

6.2.6 Results 24
6.2.7 Decision Making 25
6.2.8 Learning 26

7 Conclusion **27**

1 Introduction

When shopping in a supermarket, customers frequently encounter a recurring and practical problem: items are placed into a bag in an order that later proves to be suboptimal. For example, fragile products such as tomatoes may already be in the bag when heavier items like canned goods still need to be added. This often requires stopping, unpacking, and reorganizing the bag to prevent damage to sensitive products. Depending on the size of the shopping list, this situation can occur multiple times during a single trip. While seemingly trivial, optimizing both the packing order and the shopping route mentally becomes increasingly difficult as the number of products grows.

This challenge arises from the combined complexity of determining an efficient path through the supermarket and selecting an appropriate order in which products are placed into the shopping bag. The problem involves multiple interacting factors, including spatial distances between products, the physical properties of the items, and time constraints. Manually optimizing these aspects in real time is impractical for most shoppers.

The goal of this project is to address this problem by developing a **Supermarket Route Optimizer** that computes an optimal shopping route and product collection order. The optimization is based on three main criteria:

1. **Walking distance**, which should be minimized to reduce time spent in the supermarket and physical effort;
2. **Product hardness**, where hard products can safely be placed at the bottom of the bag, while soft or fragile products should be collected later to avoid damage;
3. **Product weight**, as heavier items should be placed below lighter ones to prevent excessive load on sensitive products.

The optimization process assumes the availability of a shopping list and a supermarket layout containing the spatial position of each product. Based on this information, the system determines a route that balances efficiency and practical feasibility, ensuring that products are collected in an order that minimizes walking distance while maintaining an appropriate packing structure. The resulting approach aims to simplify everyday shopping and allow customers to focus their attention on more important aspects of daily life.

2 Supermarket representation and Setup

2.1 Supermarket Modeling

To simulate the supermarket environment, a **grid-based model** was implemented in Python. Each cell represents a unit area of the supermarket and can be an aisle, a wall, or a shelf containing a single product category. For simplicity, each product category is assigned to **exactly one cell**, avoiding ambiguity when mapping items to locations.

The model consists of two main classes:

- **Zelle (Cell):** stores the cell's position, size, and the type of each side (aisle, wall, or shelf).
- **Grid:** manages the full layout as a 2D array of cells, with functions to create, load, save, and **plot the supermarket**. The plotting function visualizes walls, shelves, and optionally **customer paths**. Shelf sides are colored to indicate product categories, walls are drawn prominently, and paths can be overlaid as dashed lines. A legend automatically maps colors to product types.

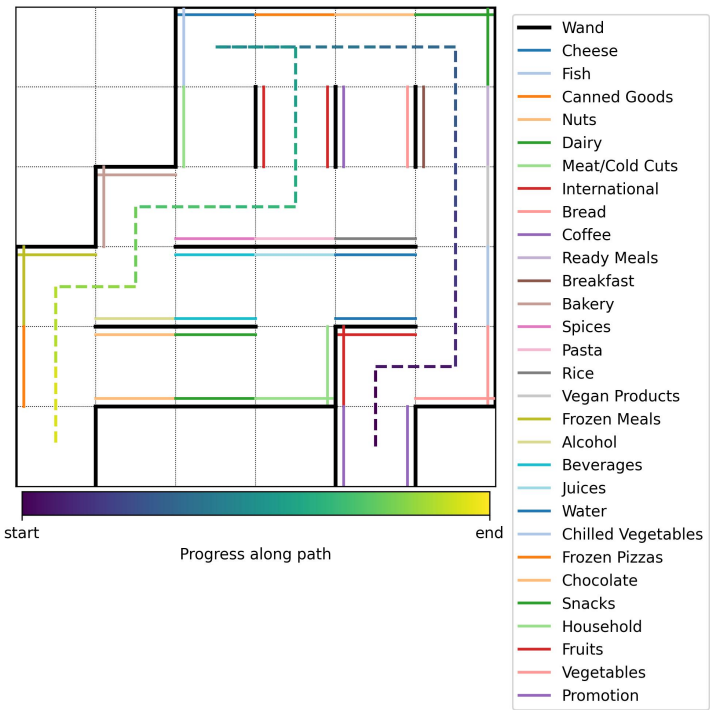

Figure 1: Example visualization of the supermarket grid. Walls are shown in black, shelves are colored by product category, and an optional customer path can be displayed as a dashed line.

This representation provides a clear, visualizable reference for each product and serves as a foundation for computing optimal shopping routes and packing strategies, considering walking distance, product weight, and hardness. Moreover, it allows for a graphical inspection of the results, making it easy to verify whether a proposed solution is reasonable and practical.

2.2 Shopping List Structure

The shopping list serves as the input for the route and packing optimization. Each item in the list is defined by four attributes:

- **Product:** the specific item to be purchased (e.g., apple, tomato).
- **Product Type:** the category of the product (e.g., fruit, dairy, canned goods).
- **Hardness:** a scale from 1 to 5 indicating the fragility of the product, where 1 is very soft and 5 is very hard.
- **Weight:** a scale from 1 to 5 representing the relative weight of the product, where 1 is light and 5 is heavy.

These attributes allow the optimization algorithm to determine not only the most efficient path through the supermarket, but also a practical order for packing items in the shopping bag, considering both fragility and weight.

3 Walking Distance Optimization

This section focuses on minimizing the total distance a customer needs to walk inside the supermarket. Efficient route planning reduces both the time spent shopping and the physical effort required. The optimization takes into account the layout of the supermarket, the location of each product, and the sequence in which items are collected. The walking-distance optimization reduces to a Traveling Salesman Problem (TSP) with fixed start and end nodes.

3.1 Walking Cost Matrix for Product Categories

As the first step in optimizing the shopping route, an algorithm was implemented to compute the walking costs between all product types in the supermarket. For this purpose, a matrix representation of path lengths was created using the *A** algorithm, which is a widely used pathfinding method capable of efficiently finding shortest paths in a grid-based environment.

The procedure consists of the following steps:

1. **Mapping product types to locations:** Each product type in the supermarket is assigned to a unique cell. Additionally, start and end positions are specified. A dictionary is created linking each product type to its corresponding cell coordinates.

2. **Matrix initialization and shortest path computation:** Each product type is assigned a unique index, and two square matrices are initialized: J for storing the walking cost (path length) between each pair of product types, and P for storing the actual path as a list of cell coordinates. For every pair of product types (including start and end), the *A** algorithm is used to compute the shortest path. The cost $J_{i,j}$ represents the walking distance from product type i to product type j, while $P_{i,j}$ contains the corresponding sequence of grid coordinates. Since a shopper can traverse paths in both directions, the cost matrix J is symmetric, i.e., $J_{i,j} = J_{j,i}$ for all i, j.

This approach allows the optimizer to have a precomputed reference of the minimal walking distances between all items, which forms the basis for further route optimization. Later, for a specific shopping list, the optimizer can simply extract the corresponding values from the matrix without needing to recompute the paths.

3.2 Product-Specific Path Matrices

Once the walking costs between product types have been computed, the relevant values for the actual products on a shopping list can be directly mapped from these matrices. Two matrices are

created for the shopping list: J_{product} for the walking costs between products and P_{product} for the corresponding paths. Each product inherits the cost and path of its product type from the precomputed matrices, so no additional pathfinding computations are required. The matrices remain symmetric, i.e., $J_{\text{product},i,j} = J_{\text{product},j,i}$, allowing straightforward extraction of distances and paths for route optimization.

	start	Cheddar	Cashews	Penne	Pizza	end
start	0	9	7	6	8	10
Cheddar	9	0	2	3	5	7
Cashews	7	2	0	3	7	9
Penne	6	3	3	0	4	6
Pizza	8	5	7	4	0	2
end	10	7	9	6	2	0

Table 1: Example product-specific cost matrix $J_{product}$

3.3 Metaheuristic Approaches for Shopping Route

The goal of the route optimization is to determine an optimal sequence of products that minimizes the total walking distance in the supermarket. Let the shopping list contain n products, indexed by $i = 1, 2, \ldots, n$, and denote the entrance as s (start) and the checkout as e (end).

Formally, the solution is a permutation π of the set $\{1, \ldots, n\}$, representing the order in which the products are collected. The total walking cost associated with a permutation π is given by

$$C(\pi) = J_{s,\pi_1} + \sum_{k=1}^{n-1} J_{\pi_k,\pi_{k+1}} + J_{\pi_n,e}, \tag{3.1}$$

where $J_{i,j}$ is the walking distance between product i and product j, as obtained from the product-specific cost matrix (Section 3.2). The objective is to find the permutation π^* that minimizes the total cost:

$$\pi^* = \arg\min_{\pi \in S_n} C(\pi), \tag{3.2}$$

where S_n is the set of all $n!$ possible permutations. Since $|S_n| = n!$, the computational complexity of a brute-force search grows factorially with n, making it infeasible for large shopping lists. For small n, however, this approach guarantees the globally optimal route.

3.3.1 Brute-Force Heuristic

As a first approach to determine an optimal shopping sequence, a brute force heuristic was implemented. All possible orders of the shopping list items (excluding start and end points) are generated,

and the total walking cost for each order is computed using the equation 3.1. The order π with the minimal total cost is selected as the optimal sequence π^*.

Product	**Product Type**
Cheddar	Käse
Cashews	Nüsse
Penne	Nudeln
Pizza	Tiefkühlgerichte
Bandnudeln	Nudeln
Blumen	Aktion

Table 2: Example short shopping list used for brute force route optimization

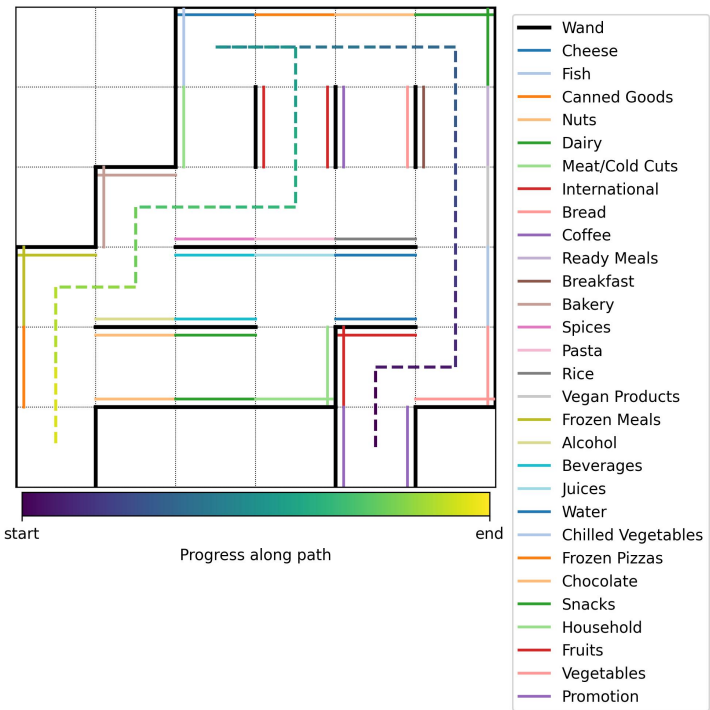

Figure 2: Optimal path π^* for a very short shopping list (table 2) computed with the brute-force heuristic

As said before, while this method guarantees a globally optimal solution for small shopping lists, its computational complexity scales factorially with the number of items ($\mathcal{O}(n!)$), making it impractical for larger lists.

Working with the brute-force heuristic taught me an important lesson about problem complexity. At first, I assumed that exhaustive search would always be impractical, but implementing it for small shopping lists revealed that, for problems of manageable size, a brute-force approach can quickly yield the true optimal solution. This experience reinforced the value of first analyzing the problem's scale and complexity before deciding on a method: sometimes the simplest approach is not only feasible, but also the most reliable.

3.3.2 Local Search Heuristic

From this point onward, a specific shopping list was used, carefully designed such that the optimal total walking cost can be visually verified to be 24. This allows for a direct comparison of heuristic results with the known optimum.

Due to the factorial growth of the brute-force approach, a more scalable heuristic based on local search was implemented. The method operates on permutations of the shopping list items while keeping the start node (0) and checkout node ($n+1$) fixed.
Given a permutation

$$\pi = (0, \pi_1, \dots, \pi_n, n+1),$$

the objective remains the minimization of the total walking cost

$$J(\pi) = \sum_{k=0}^{n} c_{\pi_k, \pi_{k+1}},$$

as defined in Equation 3.1. A neighborhood structure based on adjacent swaps was used. For a permutation of length n, $n-1$ neighboring solutions are generated by exchanging two consecutive products:

$$\pi^{(i)} = (\dots, \pi_i, \pi_{i+1}, \dots) \longrightarrow (\dots, \pi_{i+1}, \pi_i, \dots).$$

This yields a neighborhood of size $\mathcal{O}(n)$ per iteration.

Starting from a randomly generated permutation, the algorithm iteratively evaluates all neighbors and moves to the best improving one. The procedure terminates when no improving neighbor exists, i.e., at a local minimum. Since each iteration evaluates $\mathcal{O}(n)$ neighbors and each cost computation is $\mathcal{O}(n)$, the per-iteration complexity is $\mathcal{O}(n^2)$.

To mitigate convergence to poor local minima, the algorithm was executed m times with different random initial permutations (multi-start strategy). The distribution of the resulting best costs is shown in Figure 3. The histogram displays the empirical distribution, together with the mean cost and the best observed cost across all runs.

3.3.3 Extended Neighborhood: Pairwise Swaps

The neighborhood was extended to arbitrary pairwise swaps. For a shopping list of length n, this results in $\frac{n(n-1)}{2} = \mathcal{O}(n^2)$ neighboring solutions per iteration. Since each neighbor requires a full cost evaluation ($\mathcal{O}(n)$), the overall per-iteration complexity increases to $\mathcal{O}(n^3)$.

Although computationally more expensive, this richer neighborhood enables larger structural changes per iteration and reduces the risk of poor local minima. The empirical cost distribution across m multi-start runs is shown in Figure 4. Compared to the adjacent-swap variant, both the average final cost and the best obtained cost are significantly lower, despite using only 1000 instead of 10000 multi-start runs, which highlights the improved exploration capability of the extended neighborhood.

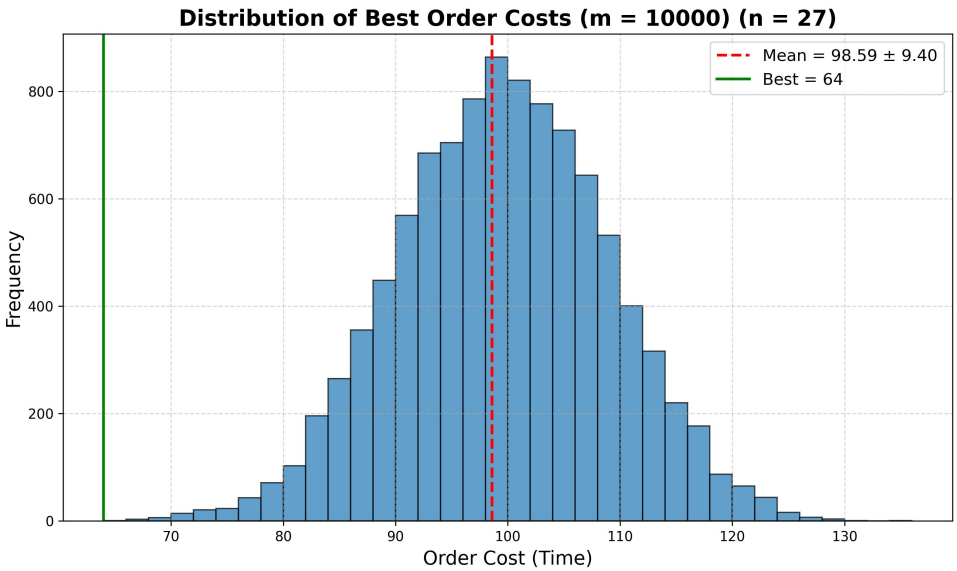

Figure 3: Distribution of best costs obtained with adjacent-swap local search. Here, n denotes the number of products in the shopping list and m the number of independent multi-start runs.

Figure 4: Distribution of best costs obtained with pairwise-swap local search. n denotes the number of products in the shopping list and m the number of independent runs.

Through implementing and testing the local search, I learned firsthand how crucial the choice of neighborhood structure is. Initially, I believed that simply running the algorithm many times would be enough to reach good solutions. However, experimenting with the extended pairwise swaps showed me that improving the way the algorithm explores the solution space can have a far greater impact on both average and best costs than sheer repetition. This insight will influence how I approach optimization problems in the future, emphasizing smarter structural design over brute-force effort.

3.3.4 GRASP (Greedy Randomized Adaptive Search Procedure)

To further enhance the quality of solutions while maintaining computational efficiency, a GRASP approach was implemented. Each iteration of GRASP consists of two phases:

1. **Greedy Randomized Construction:** Starting from the entrance, a partial order is built iteratively. At each step, the next product is selected from a Restricted Candidate List (RCL), which is constructed based on the current cost values.

   Let $C(i)$ denote the cost of selecting product i as the next element in the sequence (e.g., walking distance from the current position). Among all remaining candidates, define:

   $$C_{\min} = \min_{i \in \mathcal{R}} C(i), \tag{3.3}$$

   $$C_{\max} = \max_{i \in \mathcal{R}} C(i), \tag{3.4}$$

   where $\mathcal{R}$ is the set of remaining (unvisited) products.

   A threshold value is computed using a parameter $\alpha \in [0, 1]$:

   $$C_{\text{cutoff}} = C_{\min} + \alpha(C_{\max} - C_{\min}) \tag{3.5}$$

   The Restricted Candidate List is then defined as:

   $$\text{RCL} = \{i \in \mathcal{R} \mid C(i) \leq C_{\text{cutoff}}\} \tag{3.6}$$

   One candidate is selected uniformly at random from the RCL and appended to the partial solution. This process is repeated until a complete permutation is constructed.

   The parameter α controls the balance between greediness and randomness:

   - $\alpha = 0$: purely greedy selection (RCL contains only the best candidate),
   - $\alpha = 1$: fully random selection (all candidates included),
   - intermediate values: trade-off between intensification and diversification.

2. **Local Search Improvement:** The constructed order is then refined using the local search heuristic described in Chapter 3.3.2, exploring a neighborhood of permutations to further reduce the total walking cost.

By repeating this procedure m times, the best solution across all runs is selected. The histogram in Figure 5 illustrates the distribution of final costs, showing both the average cost (red dashed line) and the best obtained cost (green solid line). Here, n denotes the number of products in the shopping list, and m the number of GRASP iterations. The overall time complexity of the procedure depends on the chosen neighborhood structure: using the simpler neighborhood yields $\mathcal{O}(n^2)$ complexity per GRASP iteration, while a more extensive neighborhood can increase it to $\mathcal{O}(n^3)$. In my experiments, the $\mathcal{O}(n^2)$ version performed almost as well as the $\mathcal{O}(n^3)$ one in terms of solution quality.

Figure 5: Distribution of GRASP final costs over m runs for n products. Red dashed line indicates mean $\pm$ standard deviation, green solid line indicates best cost.

Implementing GRASP highlighted the power of combining randomized constructive heuristics with local search. I learned that introducing controlled randomness allows the algorithm to escape local minima that would trap a purely deterministic approach, while the local search ensures each candidate is efficiently improved. Visualizing the cost distribution confirmed that repeated runs consistently approach near-optimal solutions and provide insight into the variability of achievable outcomes.

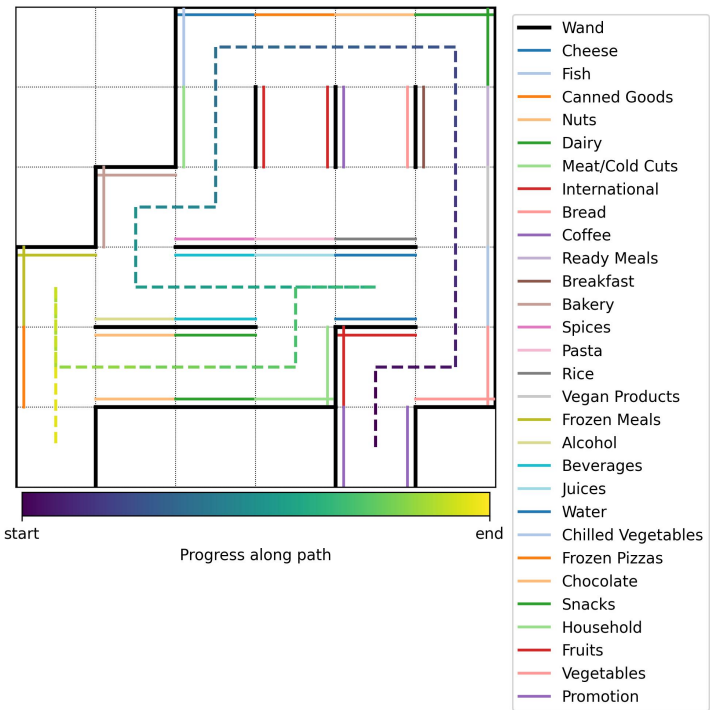

Figure 6: Example of a GRASP solution focusing on distance minimization (n = 27).

An example of a GRASP solution that optimizes only the walking distance is illustrated in Figure 6 for $n = 27$ products.

4 Product Weight Optimization

The arrangement of products in the shopping bag plays a crucial role in preventing damage during transport. In particular, heavier items should be collected earlier and placed at the bottom of the bag, ensuring that lighter items are not subjected to excessive pressure.

To formalize this idea, we introduce a cost function that penalizes undesirable orderings based on product weights. Let π denote a permutation representing the order in which products are collected. The weight of product i is denoted by w_i, where:

$$w_i \in \{1, 2, 3, 4, 5\} \tag{4.1}$$

Higher values correspond to heavier products.
The crushing cost is defined as:

$$C_{crush}(\pi) = \sum_{k=1}^{n-1} \sum_{l=k+1}^{n} \max(0, w_{\pi_l} - w_{\pi_k}) \tag{4.2}$$

This formulation evaluates all pairs of products (k, l) such that product π_k is picked before product π_l. If a heavier product is picked later (i.e., $w_{\pi_l} > w_{\pi_k}$), it will be placed on top of a lighter one, resulting in a penalty proportional to the weight difference. Otherwise, no penalty is incurred. A perfectly sorted sequence from heavy to light yields a cost of zero.

It is important to note that, when considering only this objective, finding an optimal solution is trivial. The cost function is minimized by sorting all products in non-increasing order of their weight, i.e., selecting the heaviest items first. Therefore, the problem becomes interesting only in the multi-objective setting, where weight must be balanced against other criteria such as walking distance or product fragility.

To efficiently explore the solution space, we employ a randomized greedy construction heuristic inspired by GRASP. Instead of always selecting the heaviest remaining product, we construct a Restricted Candidate List (RCL) based on a parameter $\alpha \in [0, 1]$.

At each step, let $w_{\max}$ and $w_{\min}$ denote the maximum and minimum weights among the remaining products. A cutoff threshold is defined as:

$$w_{\text{cutoff}} = w_{\max} - \alpha(w_{\max} - w_{\min}) \tag{4.3}$$

All products with weight $w_i \geq w_{\text{cutoff}}$ are included in the RCL. One of these candidates is then selected uniformly at random and appended to the order.

- For $\alpha = 0$, the method becomes purely greedy and always selects the heaviest available product.
- For $\alpha = 1$, the selection becomes completely random.
- Intermediate values of α provide a balance between greediness and diversification.

This approach biases the construction towards weight-efficient solutions while still maintaining diversity in the generated population, which is essential for effective metaheuristic optimization.

5 Product Hardness Optimization

Fragile or soft products require careful handling to avoid damage. Therefore, softer items should ideally be collected later and placed on top of more robust products in the shopping bag.

This objective is formulated analogously to the weight-based crushing cost introduced in Chapter 4. Instead of product weights, a hardness measure is used, where lower values correspond to softer (more fragile) items.

The corresponding cost function penalizes solutions in which softer products are collected too early and subsequently compressed by harder items. Thus, the structure of the objective and its optimization are conceptually identical to the weight case.

As a result, optimal solutions for this single objective can be obtained by sorting products in non-decreasing order of hardness (i.e., collecting harder items first and softer items last). Consequently, this objective becomes particularly relevant in the multi-objective setting, where trade-offs with other criteria must be considered.

6 Multi-Criteria Optimization

In many practical scenarios, optimizing a single objective is insufficient, as multiple conflicting criteria must be considered simultaneously. In this work, we focus on minimizing the walking distance, the product weight and the product hardness penalty. These objectives are inherently conflicting: minimizing distance may lead to suboptimal packing order, while strictly ordering by weight may increase travel distance.

6.1 NSGA-II for Bi-Objective Optimization

To address this trade-off, we employ the Non-dominated Sorting Genetic Algorithm II (NSGA-II), a population-based evolutionary algorithm specifically designed for multi-objective optimization.

6.1.1 Problem Formulation

Each solution is represented as a permutation π of products. Two objective functions are considered:

$$C_{\text{distance}}(\pi) \to \min \tag{6.1}$$

$$C_{\text{weight}}(\pi) \to \min \tag{6.2}$$

The goal is not to find a single optimal solution, but a set of Pareto-optimal solutions.

6.1.2 Pareto Dominance

A solution π^a is said to dominate another solution π^b if:

$$C_{\text{distance}}(\pi^a) \leq C_{\text{distance}}(\pi^b), \quad C_{\text{weight}}(\pi^a) \leq C_{\text{weight}}(\pi^b), \tag{6.3}$$

and at least one inequality is strict.
The set of all non-dominated solutions forms the Pareto front. Therefore the population is partitioned into Pareto fronts:

- Rank 1: non-dominated solutions
- Rank 2: dominated only by Rank 1
- etc.

6.1.3 Crowding Distance

To maintain diversity along the Pareto front, a crowding distance is computed for each solution. For each objective, solutions are sorted and assigned a normalized distance:

$$d_i = \sum_{m \in \{\text{distance,weight}\}} \frac{f_m(i+1) - f_m(i-1)}{f_m^{\max} - f_m^{\min}} \tag{6.4}$$

Boundary solutions are assigned an infinite crowding distance to ensure their preservation.

6.1.4 Initializing Population

A diverse initial population is crucial for the performance of evolutionary multi-objective algorithms such as NSGA-II. Instead of relying on purely random initialization, a *multi-strategy initialization* is used to balance exploration and exploitation from the beginning.

The population is constructed by combining different heuristics:

- **Random solutions (70%):** A large portion of the population is generated uniformly at random. This ensures a broad coverage of the search space and promotes diversity.
- **Weight-greedy solutions (10%):** Solutions are generated using a greedy randomized strategy that prioritizes heavier products first. At each step, a product is selected from a restricted candidate list (RCL) containing items with high weight values.
- **Distance-greedy solutions (10%):** A greedy randomized heuristic is applied to minimize walking distance. Products are selected based on proximity using a distance-based cost matrix.
- **GRASP solutions (10%):** The Greedy Randomized Adaptive Search Procedure (GRASP) is used with two different parameters:
  - 5% with $\alpha = 0.5$ (more exploratory)
  - 5% with $\alpha = 0.1$ (more greedy)

  This provides a controlled trade-off between diversification and intensification.

Formally, the initialization strategy can be written as:

$$\mathcal{P}_0 = \mathcal{P}_{\text{random}} \cup \mathcal{P}_{\text{weight}} \cup \mathcal{P}_{\text{distance}} \cup \mathcal{P}_{\text{GRASP}} \tag{6.5}$$

This hybrid initialization ensures that the initial population already contains both high-quality solutions (from greedy and GRASP methods) and diverse candidates (from random sampling), which significantly improves convergence behavior and Pareto front coverage.

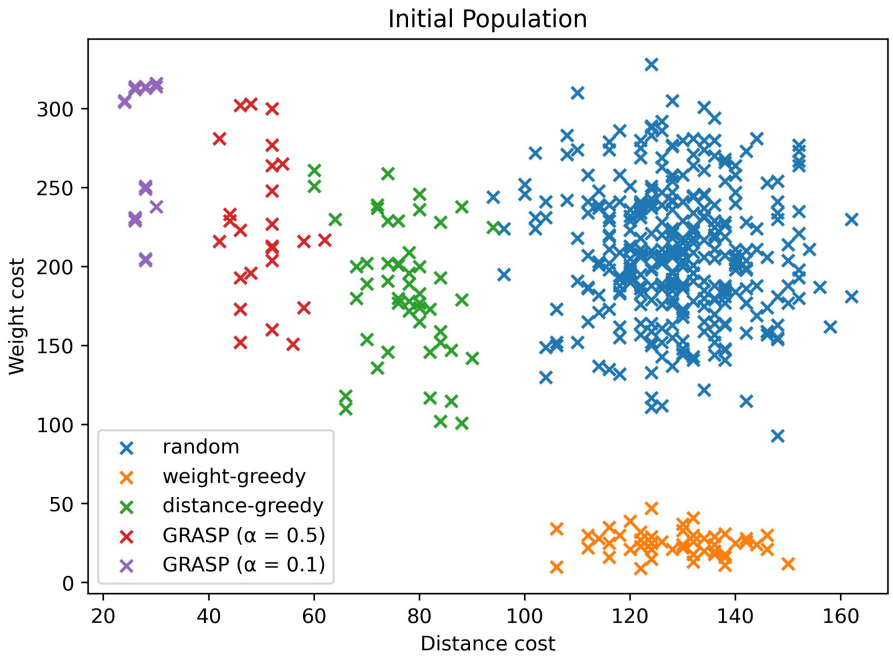

Figure 7: Distribution of the initial population in the objective space (distance vs. weight cost). Each color represents a different initialization strategy.

6.1.5 Selection, Crossover and Mutation

The evolutionary process consists of the following steps:

- **Selection:** Individuals are selected based on Pareto rank and crowding distance.
- **Crossover:** A one-point crossover is applied. The first half of one parent is combined with the remaining elements of the other parent while preserving permutation feasibility.
- **Mutation:** Random perturbations are introduced to maintain diversity.

6.1.6 Replacement Strategy

After generating offspring, the parent and child populations are merged. The next generation is formed by selecting the best individuals according to:

1. Pareto rank (ascending)
2. Crowding distance (descending)

This ensures both convergence towards the Pareto front and diversity preservation.

6.1.7 Duplicate Removal

After forming the new generation, any duplicate individuals are identified and removed. This step prevents loss of diversity.

6.1.8 Hypervolume Indicator

To evaluate convergence, the hypervolume indicator is used. For a reference point r, the dominated area is computed as:

$$HV = \sum_i (r_1 - x_i)(r_2 - y_i) \tag{6.6}$$

where (x_i, y_i) are Pareto solutions sorted by C_{distance}.
The hypervolume increases as the Pareto front improves and spreads.

Relative Hypervolume Since the absolute hypervolume depends on the choice of the reference point, a normalized version is used for better interpretability. The relative hypervolume is defined as:

$$HV_{\text{rel}} = \frac{HV}{HV_{\max}} \tag{6.7}$$

where $HV_{\max}$ denotes the maximum possible hypervolume, given by the hyper-rectangle spanned by the reference point:

$$HV_{\max} = r_1 \cdot r_2 \tag{6.8}$$

Thus, $HV_{\text{rel}} \in [0, 1]$, allowing a consistent comparison across different runs and configurations. A value close to 1 indicates that the Pareto front is both close to the optimal region and well spread.

Additionally, the relative hypervolume is used as a stopping criterion: if no significant improvement in HV_{rel} is observed over several generations, the algorithm is terminated.

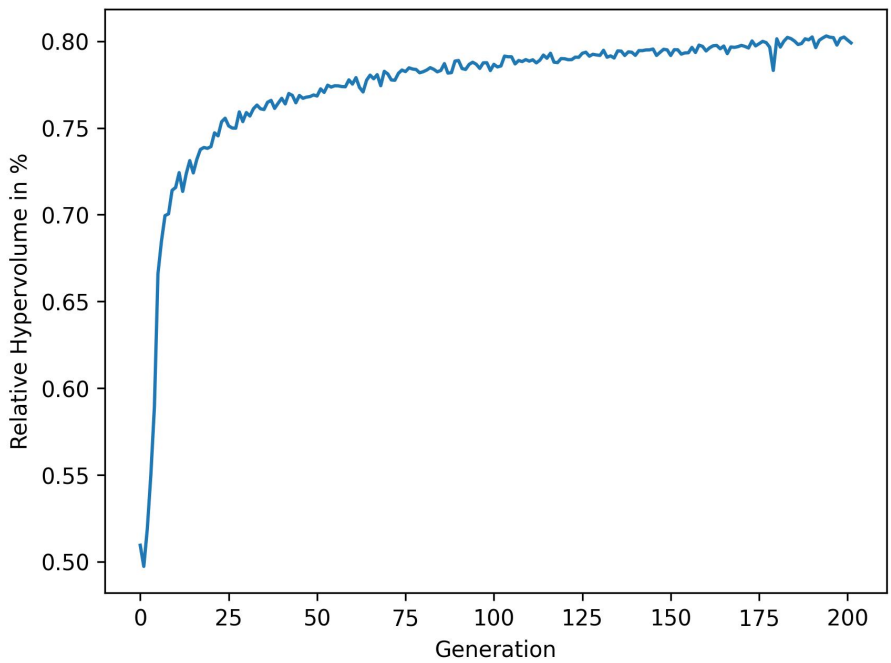

Figure 8: Evolution of the relative hypervolume over generations. Convergence is indicated by a plateau in the curve.

6.1.9 Results

After running the evolutionary algorithm, a well-distributed approximation of the Pareto front is obtained as seen in figure 9. The solutions represent different trade-offs between walking distance and product weight ordering.

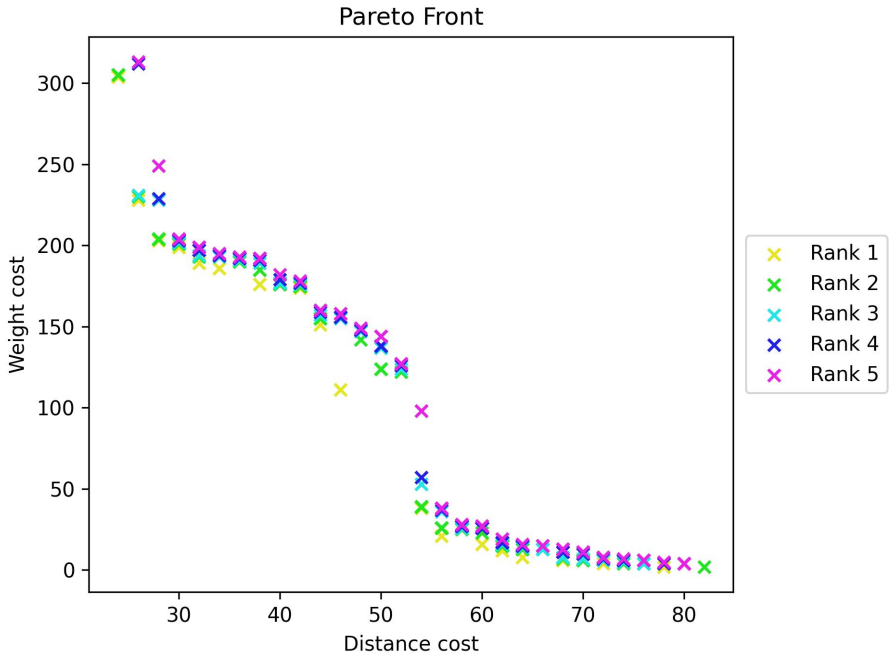

Figure 9: Final Pareto front after NSGA-II optimization (distance vs. weight cost).

It can be observed that the algorithm successfully identifies a diverse set of non-dominated solutions. Points on the left correspond to routes with minimal walking distance but potentially poor packing order, while points lower in the plot represent better weight arrangements at the cost of longer walking distances.

6.1.10 Decision Making

Once the Pareto front has been obtained, a final solution must be selected using decision aid methods. A common approach is the weighted sum method, where normalized objective values are combined using user-defined preferences.

However, it is important to note that the weighted sum method can only identify solutions that lie on the convex hull of the Pareto front. Consequently, Pareto-optimal solutions located in concave regions cannot be reached by any linear weighting. This implies that certain potentially relevant trade-off solutions are systematically excluded when relying solely on weighted aggregation. Therefore, alternative decision-making methods (e.g., reference point methods or PROMETHEE) may be required to fully explore non-convex regions of the Pareto front.

In addition, the final choice of a solution should depend on the physical packing scenario. For example, when using a shopping bag, vertical stacking dominates, making the weight objective more

critical to avoid crushing lighter items. In contrast, when using a shopping cart, more horizontal space is available, allowing heavier items to be placed next to lighter ones rather than strictly on top. As a result, the importance of the weight cost should be scaled according to the available packing area, leading to different preference weights depending on the packing context.

6.1.11 Learning

Working with the two-objective evolutionary algorithm showed that the way the initial population is created has a big impact on success. Using a mix of random, greedy, and GRASP solutions helped the algorithm find good and diverse Pareto-optimal solutions much faster. I also learned that the hypervolume is a very useful metric to evaluate performance, as it captures both the convergence and the spread of the population in the objective space.

6.2 NSGA-II for Three-Objective Optimization

The bi-objective formulation can be naturally extended by introducing an additional objective that captures product hardness. This results in a three-objective optimization problem, which is again solved using NSGA-II.

6.2.1 Problem Formulation

Each solution is represented as a permutation π of products. The following three objectives are considered:

$$C_{\text{distance}}(\pi) \to \min \tag{6.9}$$

$$C_{\text{weight}}(\pi) \to \min \tag{6.10}$$

$$C_{\text{hardness}}(\pi) \to \min \tag{6.11}$$

The hardness objective is defined analogously to the weight objective, penalizing situations where harder products are placed on top of softer ones.

6.2.2 Pareto Dominance

The concept of Pareto dominance extends directly to three objectives. A solution π^a dominates another solution π^b if:

$$C_{\text{distance}}(\pi^a) \leq C_{\text{distance}}(\pi^b), \quad C_{\text{weight}}(\pi^a) \leq C_{\text{weight}}(\pi^b), \quad C_{\text{hardness}}(\pi^a) \leq C_{\text{hardness}}(\pi^b), \tag{6.12}$$

and at least one inequality is strict. The population is again partitioned into Pareto fronts based on non-dominance, identical to the bi-objective case.

6.2.3 Crowding Distance

The crowding distance is computed analogously to the two-dimensional case, but extended to three objectives:

$$d_i = \sum_{m \in \{\text{distance,weight,hardness}\}} \frac{f_m(i+1) - f_m(i-1)}{f_m^{\max} - f_m^{\min}} \tag{6.13}$$

This ensures diversity across the entire Pareto surface.

6.2.4 Initialization, Genetic Operators and Replacement

The initialization strategy, as well as selection, crossover, mutation, replacement and duplicates removal mechanisms, remain identical to the bi-objective case. The only extension is the inclusion of hardness-based greedy solutions in the initial population to ensure sufficient diversity along the third objective.

6.2.5 Hypervolume Indicator

The hypervolume concept generalizes to three dimensions by measuring the dominated volume with respect to a reference point $r = (r_1, r_2, r_3)$. The computation is performed by decomposing the volume into slices along one objective axis.

The relative hypervolume is defined analogously as:

$$HV_{\text{rel}} = \frac{HV}{HV_{\max}}, \quad HV_{\max} = r_1 \cdot r_2 \cdot r_3 \tag{6.14}$$

and is again used as a stopping criterion.

6.2.6 Results

The resulting Pareto front forms a surface in the three-dimensional objective space, as shown in Figure 10.

Compared to the bi-objective case, the trade-offs become significantly more complex. Improvements in one objective often lead to simultaneous deterioration in two other objectives. In particular, solutions that are optimal with respect to weight are not necessarily optimal with respect to hardness, leading to a richer Pareto structure.

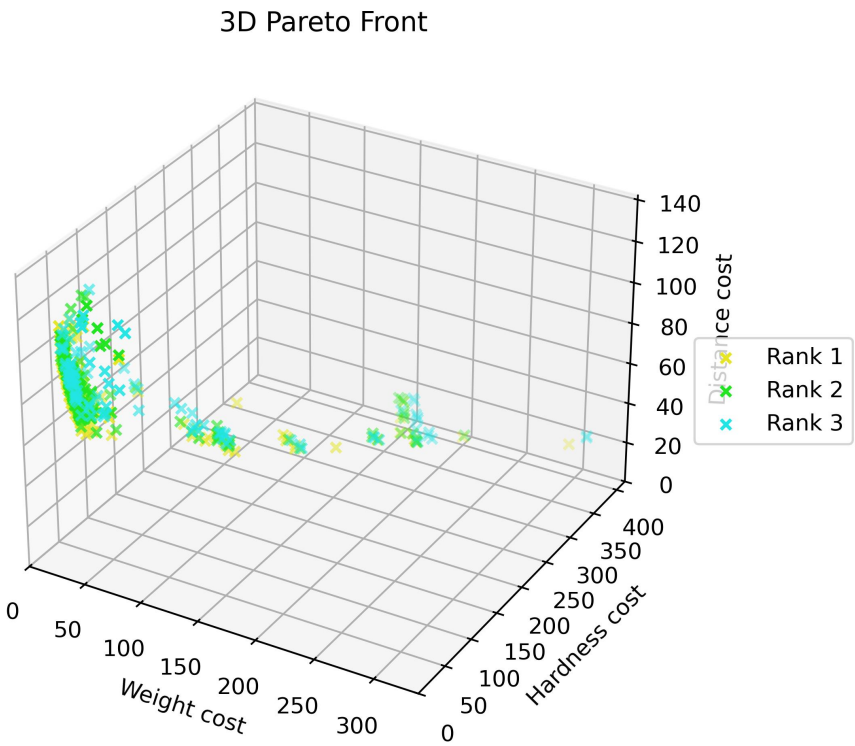

Figure 10: Final Pareto front for the three-objective problem (distance, weight, hardness).

6.2.7 Decision Making

Decision making in the three-objective case follows the same principles as in the bi-objective setting. Again one can use a weighted sum of normalized objectives to select a final solution:

$$s_i = \lambda_1 \tilde{C}_{\text{distance}}(i) + \lambda_2 \tilde{C}_{\text{weight}}(i) + \lambda_3 \tilde{C}_{\text{hardness}}(i), \tag{6.15}$$

where $(\lambda_1, \lambda_2, \lambda_3)$ are user-defined weights reflecting the relative importance of each objective.

A compromise solution from the 3-objective Pareto front can thus be selected by choosing appropriate weights. The final solution s^* is obtained by minimizing the weighted sum:

$$s^* = \arg\min_i \left(s_i\right). \tag{6.16}$$

Figure 11 shows an example of such a compromise solution with balanced trade-offs between distance, weight, and hardness.

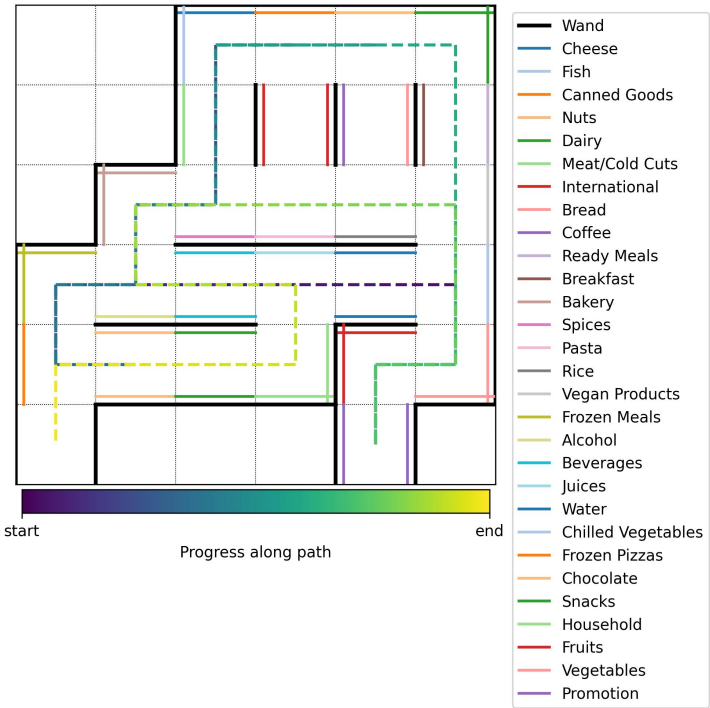

Figure 11: Example of a compromise solution on the 3-objective Pareto front (distance, weight, hardness). The displayed path corresponds to user-defined weights $(\lambda_1, \lambda_2, \lambda_3) = (0.6, 0.3, 0.1)$. The actual costs for this solution are $C_{\text{distance}} = 58$, $C_{\text{weight}} = 45$, $C_{\text{hardness}} = 63$.

6.2.8 Learning

Working with the three-objective NSGA-II revealed an important limitation of using the weighted sum method without visualizing the Pareto front. Small changes in the weights often had no effect on the selected solution, but at certain points led to sudden large jumps in all objective values. This happens because weighted sums only capture solutions on the convex parts of the Pareto front, while more balanced trade-offs in concave regions are completely missed. As a result, potentially better compromise solutions are not even considered, highlighting the importance of visualizing and understanding the Pareto structure before applying decision-making methods. Furthermore, the three-dimensional Pareto front is already significantly harder to interpret when visualized, and for problems with more than three objectives, the Pareto front can usually no longer be properly visualized. This makes the choice of a suitable decision aid method even more crucial.

7 Conclusion

In this work, I extended the classical shopping route optimization problem by incorporating additional practical objectives such as product weight and hardness. This made the problem significantly more realistic, but also highlighted the complexity that arises when multiple competing objectives must be considered simultaneously.

Through the implementation of NSGA-II, I learned how powerful evolutionary algorithms are for handling such trade-offs. Instead of searching for a single optimal solution, the algorithm provides a diverse set of Pareto-optimal solutions, allowing a deeper understanding of the structure of the problem. In particular, I observed how strongly the initialization of the population influences the final results, and how combining different heuristics can significantly improve both convergence and diversity.

For single-objective optimization, I also learned that the choice of the neighborhood structure is crucial, as it directly determines the quality of reachable solutions and the effectiveness of local search. A well-designed neighborhood can significantly improve results, while a poor choice may lead to premature convergence or suboptimal solutions.

Another important insight was the role of evaluation and decision-making. The hypervolume turned out to be a very effective metric to track progress, while the analysis of the Pareto front revealed limitations of simple methods such as weighted sums, especially in higher dimensions. This emphasized the importance of not only generating solutions, but also understanding and interpreting them correctly.

Finally, there are several additional metaheuristics that could be applied to this problem, such as Tabu Search, Simulated Annealing, or Multi-Objective Simulated Annealing. While these approaches are promising and could potentially yield further improvements, their implementation and evaluation would have gone beyond the scope of this project.

Overall, this project gave me a much better intuition for multi-objective optimization and the challenges involved in balancing conflicting goals. It also showed me that combining domain knowledge with advanced optimization techniques is key to obtaining meaningful and practical solutions.

The complete project, including all implementations and experiments, is publicly available on GitHub and can be accessed at: https://github.com/mangam-02/supermarket-route-optimizer

List of Figures

1 Example visualization of the supermarket grid. Walls are shown in black, shelves are colored by product category, and an optional customer path can be displayed as a dashed line. 6

2 Optimal path π^* for a very short shopping list (table 2) computed with the brute-force heuristic 10

3 Distribution of best costs obtained with adjacent-swap local search. Here, n denotes the number of products in the shopping list and m the number of independent multi-start runs. 12

4 Distribution of best costs obtained with pairwise-swap local search. n denotes the number of products in the shopping list and m the number of independent runs. . . . 12

5 Distribution of GRASP final costs over m runs for n products. Red dashed line indicates mean $\pm$ standard deviation, green solid line indicates best cost. 14

6 Example of a GRASP solution focusing on distance minimization (n = 27). 14

7 Distribution of the initial population in the objective space (distance vs. weight cost). Each color represents a different initialization strategy. 20

8 Evolution of the relative hypervolume over generations. Convergence is indicated by a plateau in the curve. 21

9 Final Pareto front after NSGA-II optimization (distance vs. weight cost). 22

10 Final Pareto front for the three-objective problem (distance, weight, hardness). 25

11 Example of a compromise solution on the 3-objective Pareto front (distance, weight, hardness). The displayed path corresponds to user-defined weights $(\lambda_1, \lambda_2, \lambda_3) = (0.6, 0.3, 0.1)$. The actual costs for this solution are $C_{\text{distance}} = 58$, $C_{\text{weight}} = 45$, $C_{\text{hardness}} = 63$. 26

List of Tables

1 Example product-specific cost matrix $J_{product}$ 9
2 Example short shopping list used for brute force route optimization 10